

#Exp1

```
import tensorflow as tf
a = tf.constant([1, 2, 3])
b = tf.constant([4, 5, 6])
c = tf.add(a, b)
print("Vector A:", a.numpy())
print("Vector B:", b.numpy())
print("A + B =", c.numpy())
```

#Exp2

```
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers

np.random.seed(0)

X = np.random.rand(100,1)
y = 2*X.squeeze() + 1 + np.random.randn(100)*0.1

model = keras.Sequential([
    layers.Dense(10, activation='relu', input_shape=(1,)),layers.Dense(1)
])

model.compile(optimizer='adam', loss='mse')
model.fit(X, y, epochs=100, batch_size=32, verbose=0)

pred = model.predict(X)

for i in range(5):
    print("Predicted:", pred[i][0], "\tTrue:", y[i])
```

#Exp3

import numpy as np

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

X = np.array([[0,0],[0,1],[1,0],[1,1]])

y = np.array([0,1,1,1])

model = Sequential([Dense(1, input_shape=(2,)), activation='sigmoid'])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

model.fit(X, y, epochs=1000, verbose=0)

loss, acc = model.evaluate(X, y)

print("Loss:", loss)

print("Accuracy:", acc)

print("Predictions:", model.predict(X).flatten())

Exp 4

```
import matplotlib.pyplot as plt
```

```
from tensorflow.keras.datasets import mnist
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Flatten,Dense
```

```
(X_train,y_train),(X_test,y_test)=mnist.load_data()
```

```
X_train,X_test=X_train/255.0,X_test/255.0
```

```
model=Sequential([
```

```
    Flatten(input_shape=(28,28)),
```

```
    Dense(128,activation='relu'),
```

```
    Dense(10,activation='softmax')
```

```
])
```

```
model.compile('adam','sparse_categorical_crossentropy',metrics=['accuracy'])
```

```
h=model.fit(X_train,y_train,epochs=5,validation_split=0.2)
```

```
loss,acc=model.evaluate(X_test,y_test)
```

```
print("Accuracy:",acc)
```

```
plt.plot(h.history['accuracy'],label='train')
```

```
plt.plot(h.history['val_accuracy'],label='validation')
```

```
plt.legend()
```

```
plt.title("Model Accuracy")
```

```
plt.show()
```

#Exp 5

```
import tensorflow as tf, matplotlib.pyplot as plt
```

```
from tensorflow.keras import datasets, layers, models
```

```
(X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()
```

```
X_train, X_test = X_train / 255.0, X_test / 255.0
```

```
c = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
```

```
plt.figure(figsize=(10, 10))
```

```
for i in range(25):
```

```
    plt.subplot(5, 5, i + 1)
```

```
    plt.xticks([]);
```

```
    plt.yticks([]);
```

```
    plt.grid(False)
```

```
    plt.imshow(X_train[i])
```

```
    plt.xlabel(c[y_train[i][0]])
```

```
plt.show()
```

```
model = models.Sequential([
```

```
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
```

```
    layers.MaxPooling2D(2, 2),
```

```
    layers.Conv2D(64, (3, 3), activation='relu'),
```

```
    layers.MaxPooling2D(2, 2),
```

```
    layers.Conv2D(64, (3, 3), activation='relu'),
```

```
    layers.Flatten(),
```

```
    layers.Dense(64, activation='relu'),
```

```
    layers.Dense(10)
```

```
])
```

```
model.compile('adam',
```

```
loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),  
metrics=['accuracy'])
```

```
h=model.fit(X_train,y_train,epochs=10,validation_data=(X_test,y_test))
```

```
plt.plot(h.history['accuracy'],label='train')
```

```
plt.plot(h.history['val_accuracy'],label='val')
```

```
plt.xlabel('Epoch');
```

```
plt.ylabel('Accuracy')
```

```
plt.legend();
```

```
plt.show()
```

```
print("Test accuracy:",model.evaluate(X_test,y_test,verbose=0)[1])
```

#Exp 6

import numpy as np

from tensorflow import keras

from sklearn.model_selection import train_test_split

X=np.random.rand(500,1)

y=3*X+2

x1,x2,y1,y2=train_test_split(X,y,test_size=0.2)

for e in [10,50,100]:

 m=keras.Sequential([

 keras.layers.Dense(10,activation='relu',input_shape=(1,)),

 keras.layers.Dense(1)

])

 m.compile('adam','mse')

 m.fit(x1,y1,epochs=e,verbose=0)

 print("Epochs:",e,"Loss:",m.evaluate(x2,y2,verbose=0))

#Exp 7

```
import tensorflow as tf, numpy as np
```

```
from tensorflow.keras import layers, models
```

```
from tensorflow.keras.applications import MobileNetV2
```

```
(x,y),(xt,yt)=tf.keras.datasets.cifar10.load_data()
```

```
i=np.where((y==3)|(y==5))[0]
```

```
j=np.where((yt==3)|(yt==5))[0]
```

```
x,y=x[i],(y[i]==5).astype(int)
```

```
xt,yt=xt[j],(yt[j]==5).astype(int)
```

```
x=np.array([tf.image.resize(a,(64,64)).numpy() for a in x])/255.0
```

```
xt=np.array([tf.image.resize(a,(64,64)).numpy() for a in xt])/255.0
```

```
b=MobileNetV2(input_shape=(64,64,3),include_top=0,weights='imagenet')
```

```
b.trainable=0
```

```
m=models.Sequential([b,layers.GlobalAveragePooling2D(),layers.Dense(1,activation='sigmoid')])
```

```
m.compile('adam','binary_crossentropy',metrics=['accuracy'])
```

```
m.fit(x,y,epochs=3,validation_data=(xt,yt))
```

```
p=m.predict(xt[:1])
```

```
print("Prediction:","DOG" if p[0][0]>0.5 else "CAT")
```

#Exp 11

```
import tensorflow as tf, numpy as np, matplotlib.pyplot as p
from tensorflow import keras
```

```
(x, _), _ = keras.datasets.mnist.load_data()
```

```
x = ((x - 127.5) / 127.5).reshape(-1, 28, 28, 1)
```

```
g = keras.Sequential([
    keras.layers.Dense(128, input_dim=100),
    keras.layers.LeakyReLU(0.2),
    keras.layers.Dense(784, activation='tanh'),
    keras.layers.Reshape((28, 28, 1))
])
```

```
d = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28, 1)),
    keras.layers.Dense(128),
    keras.layers.LeakyReLU(0.2),
    keras.layers.Dense(1, activation='sigmoid')
])
```

```
d.compile('adam', 'binary_crossentropy')
```

```
d.trainable = 0
```

```
gan = keras.Sequential([g, d])
```

```
gan.compile('adam', 'binary_crossentropy')
```

```
for i in range(10):
```

```
    r = x[np.random.randint(0, x.shape[0], 32)]
```

```
    z = np.random.normal(0, 1, (32, 100))
```

```
    f = g.predict(z, 0)
```

```

dl=d.train_on_batch(r,np.ones((32,1)))
d.train_on_batch(f,np.zeros((32,1)))
gl=gan.train_on_batch(z,np.ones((32,1)))
print(f"Epoch {i+1}, D Loss: {dl}, G Loss: {gl}")

img=g.predict(np.random.normal(0,1,(1,100)))
p.imshow(img.reshape(28,28),cmap='gray')
p.show()

```

#Exp12

```
import numpy as np
```

```

s=["hi","bye","thanks"]
a=["hello","goodbye","welcome"]
Q=np.zeros((3,3))
x,g,e=.1,.9,.2

```

```

def r(i,j):
    return 1 if i==j else -1

```

```

for _ in range(100):
    i=np.random.randint(3)
    j=np.random.randint(3) if np.random.rand()<e else np.argmax(Q[i])
    Q[i,j]+=x*(r(i,j)+g*np.max(Q[i])-Q[i,j])
print("Enter: hi / bye / thanks")
u=input()
if u in s:
    i=s.index(u)
    print("Bot:",a[np.argmax(Q[i])])
else:
    print("Unknown input")

```